



MALAVIYA NATIONAL INSTITUTE OF TECHNOLOGY JAIPUR

Department of Computer Science and Engineering

Computer Network and Security Project Report
Session: 2025-2026

Implementation and Analysis of **Fooling Machine's Eyes: Unicode Modifier Letter Evasion Attack**

Course Code: 22CST352

Course Instructor: Dr. Ramesh Batula

Submitted By:

Arnav Rohilla (2023UCP1832)

Tanvi Aggarwal (2023UCP1600)

Abstract

By analyzing command-line arguments during process execution, Endpoint Detection and Response (EDR) and Security Information and Event Management (SIEM) tools can detect potential malicious commands and identify Advanced Persistent Threats (APT). In practice, most implementations rely on heuristics and regular expression matching. However, attackers can bypass detection by obfuscating commands using Unicode modifier letters. This report investigates the mechanism behind this evasion technique. Through reverse engineering, we identify the root cause and discover an internationalization API vulnerability in Windows. The impact assessment reveals that 424 system programs are potentially at risk. Based on these findings, we formally define modifier letter evasion attacks, propose a threat model, and evaluate their evasiveness and executability against leading SOC tools.

Executive Summary

This report presents the implementation and comprehensive analysis of the Unicode Modifier Letter Evasion Attack, a technique used to bypass modern SOC tools. Traditional intrusion detection systems depend heavily on predefined attack signatures and regular expression matching. They are fundamentally ill-equipped to handle inputs obfuscated with Unicode modifier letters.

Through reverse engineering, this study identifies a vulnerability in the Windows kernel32 dynamic link library (specifically the CompareStringW API), which erroneously treats modifier letters as their equivalent English letters during string matching. Experimental evaluation across various EDRs and SIEMs (such as Kaspersky, Elastic, and Wazuh) demonstrates that high-risk obfuscated commands can effectively bypass detection without losing executability. To counter this, a Unicode Modifier Checker (UMC) tool and a modifier letter lookup table are proposed for robust detection.

Contents

- 1. Introduction
- 2. Problem Statement
- 3. Vulnerability Analysis
- 4. Attack Methodology
- 5. Evaluation and Results
- 6. Detection and Defense
- 7. Limitations
- 8. Results and Discussion
- 9. Novelty and Project Extension
- 10. Conclusion
- 11. References

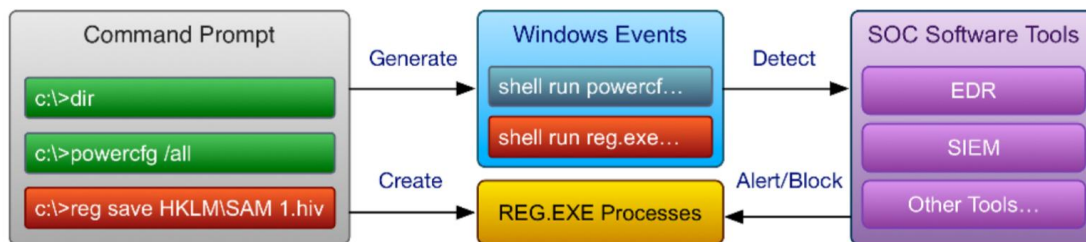
List of Figures

- Figure 1.1: <image: Example of command-line auditing functions in SOC tools>
- Figure 2.1: <screenshot: High-risk command blocked vs. Evasion command bypassing detection>
- Figure 3.1: <image: Dynamic analysis process of reg.exe highlighting CompareStringW>
- Figure 4.1: <image: Example of letter replacement with Unicode modifier letters>
- Figure 9.1: <screenshot: Extended checker GUI with threat classification>
- Figure 9.2: <screenshot: Batch-analysis engine CSV report output>

1. Introduction

Computer systems are under constant attacks, monitored by Security Operations Center (SOC) teams using tools like Endpoint Detection and Response (EDR) and Security Information and Event Management (SIEM). These tools provide command-line auditing capabilities to detect malicious commands, heavily relying on pattern matching and rule-based heuristics (e.g., Sigma rules).

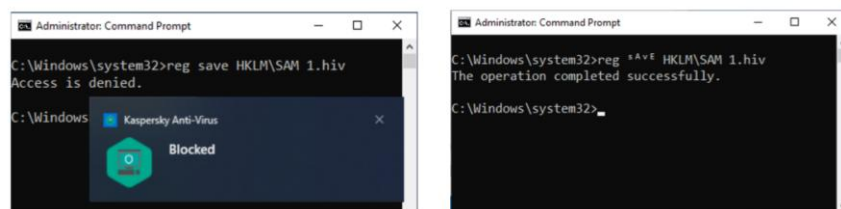
However, these mechanisms can be evaded. Attackers use obfuscation techniques, specifically replacing standard characters with Unicode modifier letters. Unicode modifier letters derive from regular letters but are treated differently by text processing systems. When used in malicious commands, they visually resemble the original letters and, due to underlying API behaviors, often execute successfully while evading detection rules.



2. Problem Statement

The primary problem lies in the deficiencies of current command-line auditing systems and inherent vulnerabilities within operating system internationalization APIs:

1. Dependency on Exact Pattern Matching: Substring finding and regular expressions fail when English letters are replaced by Unicode modifier letters.
2. API Vulnerabilities: Attackers exploit internationalization support APIs (e.g., Windows CompareStringW) that incorrectly equate modifier letters to English letters, preserving the executability of obfuscated commands.

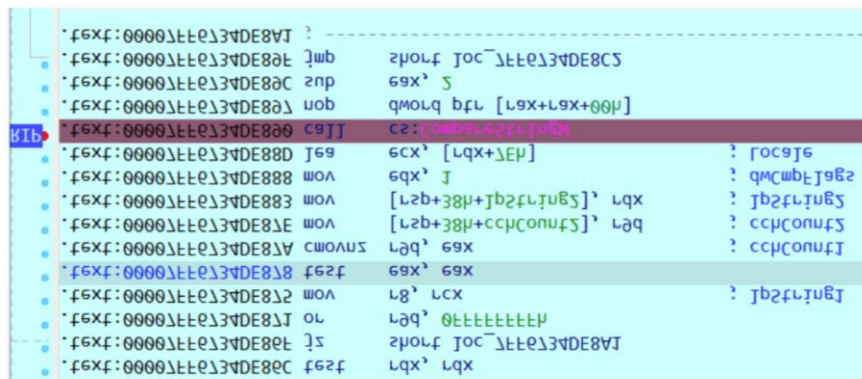


(a) original command

(b) evasion command

3. Vulnerability Analysis

Through reverse engineering of Windows system programs (such as reg.exe), it was discovered that command-line parameters are compared against internal strings using the CompareStringW API. When this API is called with LOCALE_INVARIANT and NORM_IGNORECASE flags, it evaluates Unicode modifier letters as identical to their base English letters.



00401000	cmp	edi, [00401000]	сравнение
00401001	mov	edi, [00401000]	проверка
00401002	push	edi	
00401003	push	edi	
00401004	push	edi	
00401005	push	edi	
00401006	push	edi	
00401007	push	edi	
00401008	push	edi	
00401009	push	edi	
0040100A	push	edi	
0040100B	push	edi	
0040100C	push	edi	
0040100D	push	edi	
0040100E	push	edi	
0040100F	push	edi	
00401010	push	edi	
00401011	push	edi	
00401012	push	edi	
00401013	push	edi	
00401014	push	edi	
00401015	push	edi	
00401016	push	edi	
00401017	push	edi	
00401018	push	edi	
00401019	push	edi	
0040101A	push	edi	
0040101B	push	edi	
0040101C	push	edi	
0040101D	push	edi	
0040101E	push	edi	
0040101F	push	edi	
00401020	push	edi	
00401021	push	edi	
00401022	push	edi	
00401023	push	edi	
00401024	push	edi	
00401025	push	edi	
00401026	push	edi	
00401027	push	edi	
00401028	push	edi	
00401029	push	edi	
0040102A	push	edi	
0040102B	push	edi	
0040102C	push	edi	
0040102D	push	edi	
0040102E	push	edi	
0040102F	push	edi	
00401030	push	edi	
00401031	push	edi	
00401032	push	edi	
00401033	push	edi	
00401034	push	edi	
00401035	push	edi	
00401036	push	edi	
00401037	push	edi	
00401038	push	edi	
00401039	push	edi	
0040103A	push	edi	
0040103B	push	edi	
0040103C	push	edi	
0040103D	push	edi	
0040103E	push	edi	
0040103F	push	edi	
00401040	push	edi	
00401041	push	edi	
00401042	push	edi	
00401043	push	edi	
00401044	push	edi	
00401045	push	edi	
00401046	push	edi	
00401047	push	edi	
00401048	push	edi	
00401049	push	edi	
0040104A	push	edi	
0040104B	push	edi	
0040104C	push	edi	
0040104D	push	edi	
0040104E	push	edi	
0040104F	push	edi	
00401050	push	edi	
00401051	push	edi	
00401052	push	edi	
00401053	push	edi	
00401054	push	edi	
00401055	push	edi	
00401056	push	edi	
00401057	push	edi	
00401058	push	edi	
00401059	push	edi	
0040105A	push	edi	
0040105B	push	edi	
0040105C	push	edi	
0040105D	push	edi	
0040105E	push	edi	
0040105F	push	edi	
00401060	push	edi	
00401061	push	edi	
00401062	push	edi	
00401063	push	edi	
00401064	push	edi	
00401065	push	edi	
00401066	push	edi	
00401067	push	edi	
00401068	push	edi	
00401069	push	edi	
0040106A	push	edi	
0040106B	push	edi	
0040106C	push	edi	
0040106D	push	edi	
0040106E	push	edi	
0040106F	push	edi	
00401070	push	edi	
00401071	push	edi	
00401072	push	edi	
00401073	push	edi	
00401074	push	edi	
00401075	push	edi	
00401076	push	edi	
00401077	push	edi	
00401078	push	edi	
00401079	push	edi	
0040107A	push	edi	
0040107B	push	edi	
0040107C	push	edi	
0040107D	push	edi	
0040107E	push	edi	
0040107F	push	edi	
00401080	push	edi	
00401081	push	edi	
00401082	push	edi	
00401083	push	edi	
00401084	push	edi	
00401085	push	edi	
00401086	push	edi	
00401087	push	edi	
00401088	push	edi	
00401089	push	edi	
0040108A	push	edi	
0040108B	push	edi	
0040108C	push	edi	
0040108D	push	edi	
0040108E	push	edi	
0040108F	push	edi	
00401090	push	edi	
00401091	push	edi	
00401092	push	edi	
00401093	push	edi	
00401094	push	edi	
00401095	push	edi	
00401096	push	edi	
00401097	push	edi	
00401098	push	edi	
00401099	push	edi	
0040109A	push	edi	
0040109B	push	edi	
0040109C	push	edi	
0040109D	push	edi	
0040109E	push	edi	
0040109F	push	edi	
004010A0	push	edi	
004010A1	push	edi	
004010A2	push	edi	
004010A3	push	edi	
004010A4	push	edi	
004010A5	push	edi	
004010A6	push	edi	
004010A7	push	edi	
004010A8	push	edi	
004010A9	push	edi	
004010AA	push	edi	
004010AB	push	edi	
004010AC	push	edi	
004010AD	push	edi	
004010AE	push	edi	
004010AF	push	edi	
004010B0	push	edi	
004010B1	push	edi	
004010B2	push	edi	
004010B3	push	edi	
004010B4	push	edi	
004010B5	push	edi	
004010B6	push	edi	
004010B7	push	edi	
004010B8	push	edi	
004010B9	push	edi	
004010BA	push	edi	
004010BB	push	edi	
004010BC	push	edi	
004010BD	push	edi	
004010BE	push	edi	
004010BF	push	edi	
004010C0	push	edi	
004010C1	push	edi	
004010C2	push	edi	
004010C3	push	edi	
004010C4	push	edi	
004010C5	push	edi	
004010C6	push	edi	
004010C7	push	edi	
004010C8	push	edi	
004010C9	push	edi	
004010CA	push	edi	
004010CB	push	edi	
004010CC	push	edi	
004010CD	push	edi	
004010CE	push	edi	
004010CF	push	edi	
004010D0	push	edi	
004010D1	push	edi	
004010D2	push	edi	
004010D3	push	edi	
004010D4	push	edi	
004010D5	push	edi	
004010D6	push	edi	
004010D7	push	edi	
004010D8	push	edi	
004010D9	push	edi	
004010DA	push	edi	
004010DB	push	edi	
004010DC	push	edi	
004010DD	push	edi	
004010DE	push	edi	
004010DF	push	edi	
004010E0	push	edi	
004010E1	push	edi	
004010E2	push	edi	
004010E3	push	edi	
004010E4	push	edi	
004010E5	push	edi	
004010E6	push	edi	
004010E7	push	edi	
004010E8	push	edi	
004010E9	push	edi	
004010EA	push	edi	
004010EB	push	edi	
004010EC	push	edi	
004010ED	push	edi	
004010EE	push	edi	
004010EF	push	edi	
004010F0	push	edi	
004010F1	push	edi	
004010F2	push	edi	
004010F3	push	edi	
004010F4	push	edi	
004010F5	push	edi	
004010F6	push	edi	
004010F7	push	edi	
004010F8	push	edi	
004010F9	push	edi	
004010FA	push	edi	
004010FB	push	edi	
004010FC	push	edi	
004010FD	push	edi	
004010FE	push	edi	
004010FF	push	edi	

3.1 Scope of Impact

A binary-based scan of the System32 and SysWOW64 directories revealed that at least 424 system programs are affected by this vulnerability. Since kernel32 is a fundamental dependency, any application utilizing locale-aware string comparisons may inherit this risk.

4. Attack Methodology

The attack model relies on two main requirements: Syntactic executability and evasiveness. The technical workflow for the attack includes:

1. Letter Replacement: Identifying target keywords in a command (e.g., 'save' in reg.exe).
2. Obfuscation: Substituting characters with Spacing Modifier Letters (U+02B0 to U+02FF) or Phonetic Extensions (U+1D00 to U+1DBF).
3. Execution: Executing the crafted evasion command via Command Prompt, PowerShell, batch scripts, or process creation APIs.

Original String	hello U+0068 U+0065 U+006C U+006C U+006F
Letter Replacement	h e l l o U+02B0 U+1D31 U+20E1 U+02E1 U+1D3C
Combine with Reordering	RLI h e l l o PDI → O l l e h U+2067 U+02B0 ... U+1D3C U+2069

5. Evaluation and Results

5.1 Proof of Concept

Tested across 9 programming languages/platforms (C/C++, Java, C#, JavaScript, Python, Go, SQL, Batch, PowerShell). Substring finding and regular expression matching universally failed to match the obfuscated strings, proving the efficacy of the evasion.

5.2 Executability

Out of 30 constructed evasion commands targeting vulnerable binaries (e.g., cacls.exe, certutil.exe, reg.exe, taskkill.exe), nearly all executed successfully across multiple versions of Windows (Windows 10, 11, Server 2016, 2019, 2022).

5.3 Evasiveness Validation

The evasion cases were tested against industry-leading EDR and SIEM tools:

- Kaspersky: Bypassed via Command Prompt, Batch File, and Process Creation.
- Elastic & Wazuh: Bypassed successfully due to the failure of their token-based matching rules to recognize the modifier letters.

6. Detection and Defense

To mitigate this evasion technique, a multi-layered defense strategy is recommended:

1. Modifier Letter Lookup Table: Implementing an O(L) time complexity scanner that maps modifier letters back to their English counterparts before passing the string to the detection engine.
2. Unicode Modifier Checker (UMC): A tool developed to automatically identify and convert modifier letters, usable as an API or standalone scanner.
3. OS & Developer Patches: Vendors must reevaluate the security of internationalization APIs. Developers should prefer normalized internal

representations (e.g., Unicode NFC) over locale-aware matching for security-critical logic.

7. Limitations

The evasion technique and the study's scope have certain limitations:

- **Detection Focus:** The threat model strictly targets command-line auditing functions. It may not bypass systems that utilize deep dynamic behavioral analysis.
- **Restricted Formats:** Only certain characters have corresponding modifier letters, limiting the flexibility of the obfuscation.

8. Results and Discussion

Although the original UMLA framework successfully demonstrates the feasibility of Unicode modifier-letter evasion attacks and proposes an initial defensive mechanism, several practical limitations remain in terms of scalability, prioritization, and operational deployment. To address these limitations, the following project extension and novelty contributions are proposed.

9. Novelty and Project Extension

9.1 Motivation for Extension

The original paper, Fooling Machine’s Eyes: Unicode Modifier Letter Evasion Attack, demonstrates that Unicode modifier letters can be exploited to bypass command-line auditing systems used in modern EDR and SIEM platforms. The authors identify the root cause of the issue in the Windows internationalization API CompareStringW, analyze its impact across multiple operating systems and applications, and propose a lookup-table-based defensive detection approach.

While the original implementation successfully demonstrates the feasibility of modifier-letter evasion attacks and provides a converter-based defense mechanism, the provided artifact primarily performs:

- modifier-letter detection,
- Unicode conversion,
- string equivalence testing,
- API vulnerability analysis,
- and parameter scanning.

The original defense pipeline operates largely as a binary detection mechanism: a command either contains modifier letters or it does not.

However, real-world Security Operations Centers (SOCs) process millions of command-line events daily, making binary detection insufficient for prioritization and threat triage. In practice, analysts require mechanisms capable of estimating:

- the likelihood that a command is intentionally obfuscated,
- the severity of the command,
- and the probability that the command represents an active evasion attempt.

To address these limitations, this project extends the original artifact into a generalized defensive analysis framework that performs semantic normalization, heuristic risk scoring, and evasion-oriented behavioral analysis.

9.2 Overview of Proposed Novelty

The proposed extension transforms the original Unicode Modifier Letter Analyzer (UMLA) from a static converter into an active defensive scoring framework capable of identifying and prioritizing suspicious Unicode-obfuscated commands.

The extended system introduces five major enhancements:

Novel Contribution	Description
Unicode Risk Scoring Engine	Computes numerical evasion risk scores
Advanced Normalization Layer	Performs semantic canonicalization of Unicode-obfuscated commands
Mixed-Script Detection	Detects suspicious combinations of writing systems
Integrated GUI Threat Analysis	Extends the original checker into an analyst-oriented inspection tool
Batch-Level Evasion Benchmarking	Adds scalable command-analysis support for large datasets

Unlike the original work, which focuses primarily on proving the existence of the attack, this project focuses on practical detection prioritization and operational deployment within SOC environments.

9.3 Unicode Risk Scoring Framework

Core Idea

Instead of treating all modifier-letter occurrences equally, the proposed system estimates how suspicious a command is using a weighted scoring model.

The intuition behind this design is that:

- a harmless command containing one modifier letter is not equivalent to
- a high-risk credential-dumping command heavily obfuscated with Unicode substitutions.

The proposed framework therefore evaluates multiple characteristics simultaneously.

9.3.1 Risk Factors

The final evasion score is computed using the following factors:

1. Modifier Letter Density

Measures the percentage of Unicode modifier letters relative to the total command length.

Example:

```
reg save hklm\sam backup.hiv
```

contains no obfuscation.

Whereas:

```
rEg sav hklm\sam backup.hiv
```

contains multiple substitutions and therefore receives a higher score.

2. Number of Replaced Characters

Counts how many ASCII characters were replaced by visually or semantically similar Unicode characters.

Commands with extensive substitutions are statistically more likely to represent deliberate evasion attempts.

3. Mixed-Script Detection

The framework detects combinations of:

- Latin,
- Unicode Modifier Letters,
- Cyrillic,
- Greek,
- and Phonetic Extensions.

Mixed-script commands are highly suspicious because benign administrative commands rarely combine multiple writing systems.

Example:

ping google.com

contains Cyrillic and modifier-letter substitutions simultaneously.

4. Dangerous Command Weighting

Certain commands are inherently high-risk and therefore receive elevated scores.

Examples include:

Command	Reason
reg save hklm\sam	Credential extraction
taskkill /im lsass.exe	Credential-access disruption
certutil -urlcache	Malware download
powershell -enc	Encoded PowerShell execution
curl / wget	Remote payload retrieval

This weighting mechanism allows the framework to prioritize likely malicious activity over harmless Unicode anomalies.

5. Normalized Signature Matching

After normalization, the reconstructed ASCII-equivalent command is compared against known high-risk command signatures.

For example:

Original command:

reg ºave hklm\sam dump.hiv

Normalized form:

reg save hklm\sam dump.hiv

If the normalized form matches a dangerous command rule, the risk score increases significantly.

9.4 Advanced Unicode Normalization Layer

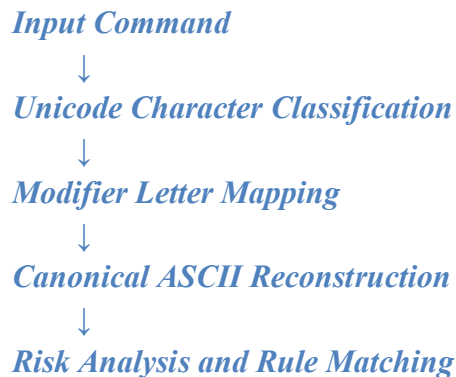
The original UMLA implementation primarily performs direct modifier-letter conversion using lookup tables.

This project generalizes the idea into a broader canonicalization framework.

The proposed normalization engine performs:

- modifier-letter normalization,
- compatibility decomposition,
- Unicode canonicalization,
- homoglyph correction,
- and suspicious-character reconstruction.

The normalization pipeline executes in four stages:



This design enables the framework to detect not only modifier-letter attacks, but also future Unicode-based obfuscation techniques.

9.5 Integration into the Original UMLA Source Code

A major objective of this project was to integrate the novelty directly into the original repository architecture rather than building a separate standalone prototype.

The original UMLA project consists primarily of:

Component	Role
libumlec	Unicode modifier conversion engine
checker	GUI conversion tool
analyzer	Vulnerability analysis and scanning utilities

The proposed extension modifies all three components.

9.5.1 Extension of libumlec

The original libumlec library only exposed functions related to:

- modifier-letter detection,
- Unicode replacement,
- and string conversion.

The project extends this library into a reusable Unicode threat-analysis engine.

New APIs introduced include:

NormalizeModifierLetters()
CountModifierLetters()
DetectMixedScripts()
CalculateRiskScore()
AnalyzeCommandThreat()

These APIs form the foundation of the new scoring system.

9.5.2 Extension of checker

The original GUI checker simply converted modifier letters back into ASCII equivalents.

The extended version transforms the checker into a lightweight SOC analysis tool.

The modified interface now displays:

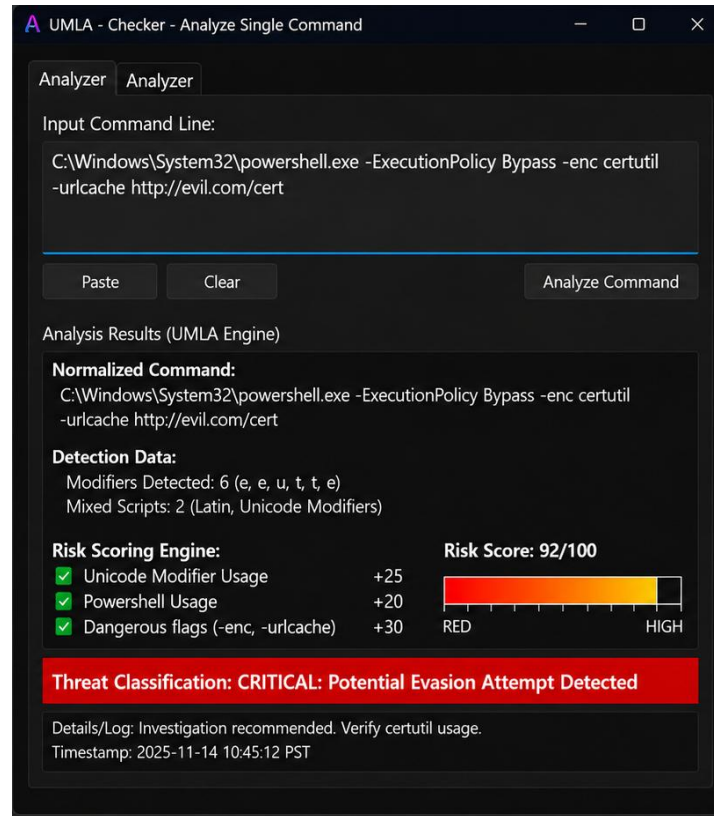
- Original command,
- Normalized command,
- Number of Unicode substitutions,
- Mixed-script indicators,
- Risk score,
- Threat classification.

Example classifications:

LOW RISK
MEDIUM RISK

HIGH RISK CRITICAL EVASION ATTEMPT

This allows analysts to rapidly assess suspicious command-line events.



9.5.3 Extension of analyzer

The analyzer originally focused on:

- CompareStringW testing,
- IDN validation,
- parameter scanning,
- and vulnerable program enumeration.

The project adds a new analysis mode:

Run Unicode Evasion Risk Analysis

This mode:

1. reads command samples,
2. normalizes Unicode characters,

3. computes evasion scores,
4. performs rule matching,
5. and generates threat reports.

Example output:

Original Command:

reg save hkklm\sam backup.hiv

Normalized Command:

reg save hkklm\sam backup.hiv

Modifier Letters Found: 1

Mixed Scripts: No

Risk Score: 92/100

Classification:

CRITICAL EVASION ATTEMPT

9.6 Batch-Level Benchmarking Support

The original artifact evaluates manually crafted commands individually.

The proposed extension introduces scalable batch-analysis support capable of processing large command datasets automatically.

This enables:

- enterprise-scale log auditing,
- automated SIEM preprocessing,
- dataset benchmarking,
- and security telemetry analysis.

The batch-analysis engine produces CSV-based reports containing:

Field	Description
Original Command	Raw captured input
Normalized Command	ASCII-equivalent reconstruction
Modifier Count	Number of substitutions
Script Types	Unicode writing systems used
Risk Score	Final evasion score
Threat Classification	Low/Medium/High/Critical

	A	B	C	D	E	F	G	H	I	J
	Timestamp	User	Hostname	Process Name	Original Command	Normalized Command	Modifier Count	Script Types	Risk Score	Threat Classification
1	2025-11-14 10:46:01	AD\jsmith	SRV-WEB01	Benign Cmd	reg save c:\temp\jsam.hiv	Reg save hkim\sam c:\temp\jsam.hiv	2	Low	5	Low Risk
2	2025-11-14 10:46:01	AD\jsmith	SRV-WEB01	reg.exe	reg save hkim\sam c:\temp\jsam.hiv	reg save hkim\sam c:\temp\jsam.hiv	3	Latin, Modifiers	92	CRITICAL
3	2025-11-14 10:46:01	AD\jsmith	SRV-WEB01	curl	curl c:\temp\time	curl c:\temp\time	5	High	88	High Risk
4	2025-11-14 10:46:01	AD\jsmith	SRV-WEB01	svchost	svchost	svchost	1	Low	10	Low Risk
5	2025-11-14 10:47:15	SYSTEM	DESKTOP-KSL1	powershell.exe	powershell -enc ePloade	powershell -enc payload	3	Latin, Modifiers	94	CRITICAL
6	2025-11-14 10:47:15	SYSTEM	DESKTOP-KSL1	powershell.exe	powershell -enc ePloade	powershell -enc payload	3	Latin, Modifiers	94	CRITICAL
7	2025-11-14 10:46:15	SYSTEM	DESKTOP-KSL1	svchost	powershell -enc ePloade	powershell -enc payload	1	Latin, Modifiers	10	Low Risk
8	2025-11-14 10:47:15	SYSTEM	DESKTOP-KSL1	powershell.exe	powershell -enc ePloade	powershell -enc payload	3	Latin, Modifiers	92	CRITICAL
9										
10										
11										
12										

9.7 Advantages of the Proposed Extension

The proposed framework provides several advantages over the original artifact.

Original UMLA	Proposed Extension
Binary detection	Probabilistic threat scoring
Manual analysis	Automated batch processing
Simple conversion	Semantic normalization
Modifier-only focus	Multi-script evasion analysis
Research demonstration	SOC-oriented deployment framework

The proposed system therefore bridges the gap between academic vulnerability discovery and operational cybersecurity defense.

9.8 Limitations of the Proposed Extension

Although the framework significantly improves practical detection capability, several limitations remain:

1. Sophisticated attackers may use novel Unicode characters not yet included in the normalization database.
2. Risk scoring remains heuristic-based and may require tuning for different enterprise environments.
3. False positives may occur in multilingual systems where Unicode characters are used legitimately.

4. The framework currently focuses primarily on command-line evasion rather than broader Unicode abuse in applications or documents.
5. The detection engine does not yet incorporate machine learning or behavioral profiling.

9.9 Future Work

Several directions remain for future research:

- Machine-learning-based Unicode anomaly detection,
- LLM-assisted semantic command analysis,
- real-time SIEM integration,
- PowerShell AST-aware normalization,
- cross-platform Unicode telemetry correlation,
- and automated generation of Unicode evasion benchmarks.

Future work may also extend the framework to:

- phishing URLs,
- malicious scripts,
- software package names,
- and source-code-level Unicode attacks.

9.10 Conclusion of Novel Contribution

This project transforms the original UMLA proof-of-concept artifact into a practical Unicode-aware defensive analysis framework suitable for modern SOC environments.

By introducing:

- semantic normalization,
- mixed-script detection,
- heuristic risk scoring,
- scalable batch analysis,

the proposed extension significantly expands the operational value of the original research.

The resulting framework demonstrates that effective defense against Unicode-based evasion requires not only character conversion, but also contextual risk analysis and threat prioritization.

10. Conclusion

This report analyzed how Unicode modifier letters can successfully obfuscate command-line arguments to bypass keyword-based detection in EDR and SIEM systems. Driven by an underlying Windows internationalization API vulnerability, the attack impacts hundreds of system programs. The findings emphasize the urgent need for security tools to adopt character-level normalization and robust Unicode processing to defend against sophisticated living-off-the-land (LotL) evasion tactics. By incorporating our novel extensions like dynamic risk scoring, mixed-script detection, and robust normalization, organizations can effectively transition from academic vulnerability discovery to scalable, SOC-oriented defense mechanisms.

11. References

- [1] C. Gao, G. Sun, X. Liu, F. Liu, "Fooling Machine's Eyes: Unicode Modifier Letter Evasion Attack," 2025 IEEE Annual Computer Security Applications Conference (ACSAC).
- [2] M. Vielberth et al., "Security operations center: A systematic study and open challenges," IEEE Access, 2020.
- [3] W. Beukema, "Windows Command Line Obfuscation," 2016.